

NAAN MUDHALVAN PROJECT DOCUMENTATION

Full Stack Development (MERN) Architecture Layout Blueprint

Date	October 03,2026
Team ID	SWTID-2026-7210
Project Title	Script Controlled ACL: Restrict record access based on field value
Domain Focus	Full Stack Development with MERN (MongoDB, Express, React, Node.js)

1. Introduction

- **Project Title:** Script Controlled ACL: Restrict record access based on field value
- **Team Members:**

Team Member	Role
Merlin K (TL)	Import Data & Testing
Sahana G	Data Validation & Documentation
Saraswathi K	Transform Map & Data Mapping
Nithya Sree A	Project Development & Integration
Priya Dharshini N	Testing & Documentation

2. Project Overview

- **Purpose:** To design and deploy a secure middleware engine that intercepts API payloads and dynamically acts as an access gateway to restrict row-level database visibility according to real-time object state configurations (e.g., Status == 'Confidential') rather than rigid static role groupings.
- **Features:** Context-aware API authorization loops, sandboxed server-side criteria script evaluator compiler, comprehensive React policy-mapping rules administrator control panel, and asynchronous secure cryptographic exception tracking logger pipeline.

3. Architecture

- **Frontend Architecture:** React.js single-page modular dashboard built with declarative routing structures and state controllers using Axios hooks to sync real-time database visibility state policies.
- **Backend Architecture:** Node.js and Express.js decoupled secure server engine operating asynchronous routing gateway pipelines to intercept client payload fetches and apply context validations before compilation.
- **Database Architecture:** MongoDB Atlas non-relational structural cluster housing explicit policy schemas, transaction models, and operational database collections using schema validation matrices.

4. Setup Instructions

- **Prerequisites:** Node.js LTS environment runtime core setup, MongoDB Atlas Cloud / Local daemon connection instance, npm repository package manager installer tool.
- **Installation Guide:** 1. Clone project repository link.
2. Execute 'npm install' inside root directories to pull down vm sandboxes, dependencies, and token authorization cores.
3. Configure local hidden environment variables (.env files containing MONGODB_URI and JWT_SECRET keys).

5. Folder Structure

- **Client Structure:** /client

```
|— /src
|   |— /components (UI Views, Policy Grid Forms)
|   |— /hooks (Axios API connection intercepts)
|   |— App.js (State controller routing panel)
```

- **Server Structure:** /server

```
|— /config (Database connection daemons)
|— /middleware (Script-Controlled ACL intercept engines)
|— /models (MongoDB document schemas collection definitions)
|— server.js (Express app root initializing gates)
```

6. Running the Application

- **Local Server Execution:** Commands to bootstrap runtime nodes:
 - Frontend: Run 'npm start' command lines executed cleanly inside the /client subdirectory framework.
 - Backend: Run 'npm start' command lines executed cleanly inside the /server root workspace.

7. API Documentation

No custom rest API is Required for the basic implementation of this Project.

8. Authentication & Authorization

- **Token Security Framework:** Secured using JSON Web Tokens (JWT) mapped securely within active browser sessionStorage contexts. Backend middleware extracts payload credentials parameters alongside live target record strings to evaluate clearance rules.

9. User Interface

- **Visualization Panels Descriptions:** Dedicated code validation views block area including interactive status tracking matrix rows, green text execution success status parameters, and warning alert grids indicating row access failures.

10. Testing Strategy

- **Validation Lifecycle Tools:** Functional and structural execution vectors tested via Mocha and Chai frameworks, concurrent route validation stress evaluations triggered through load scripts, and error loop sandboxing verification metrics analysis.

11. Screenshots or Demo Layout

- **Operational Workspace Visuals:** [Functional Image Placeholders Mapping Active React Interface, Express Gateway Console Routing Logs, and Filtered Database Documents Pipelines Outputs]

12. Known Issues & Boundaries

- **System Boundary Anomalies:** Minor memory mapping calculation latency jumps during extremely deep nested documents evaluation processes without active Redis pipeline synchronization layers.

13. Future Enhancements

- **Scaling Roadmaps Roadmap:** Integrating predictive artificial intelligence machine learning nodes to automatically categorize sensitive fields labels context dynamically across unstructured documents grids blocks.